```
    )
  }
  return appSubscription
}
```

This is mostly the same initSubscription code that we used before. The only difference is that when it receives ticket data, it dispatches to the same setTickets action that we wrote for the useEffect when the page loads. We don't even have to convert the JSON; ActionCable has already done so.

And then the new context is used like this:

chapter_11/02/app/javascript/components/app.tsx

```
return (
  <VenueContext.Provider value={{ state, dispatch }}>
    <SubscriptionContext.Provider
      value={initSubscription(state, dispatch)}>
      <Venue />
    </SubscriptionContext.Provider>
  </VenueContext.Provider>
)
```

Here we create a new context whose data is just an ActionCable subscription, and we nest that context inside our existing VenueContext so it is both able to use state and dispatch and is global to the rest of the components. (Note that a couple of items need to be imported at the top of the file for this to work.)

I'll be honest: this feels a little like string and sealing wax, and it wasn't the first thing I tried, but it does work, and it is mostly transparent to the rest of the internals. Please note that we still need to keep a global variable around to prevent the subscription from being created multiple times—I am not completely sure why the value prop is evaluated over and over again, but experimentation shows that it sure is.

We also need to tweak the server side (in ConcertChannel) so that it sends down the same set of ticket data, Ticket.data_for_concert(data["concertId"]), that we used in the TicketController#index method.